

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №6
По дисциплине: «ОМО»
Тема: «Рекуррентные нейронные сети»

Выполнил:
Студент 3-го курса
Группы АС-66
Осовец А.О.
Проверил:
Крощенко А.А.

Брест 2025

Вариант 7

1. Выполнить моделирование прогнозирующей нелинейной ИНС. Для генерации обучающих и тестовых данных использовать функцию. Для прогнозирования использовать многослойную ИНС с одним скрытым слоем. В качестве функций активации для скрытого слоя использовать сигмоидную функцию, для выходного - линейную.

№ варианта	a	b	c	d	Кол-во входов ИНС	Кол-во НЭ в скрытом слое
1	0.1	0.1	0.05	0.1	6	2
2	0.2	0.2	0.06	0.2	8	3
3	0.3	0.3	0.07	0.3	10	4
4	0.4	0.4	0.08	0.4	6	2
5	0.1	0.5	0.09	0.5	8	3
6	0.2	0.6	0.05	0.6	10	4
7	0.3	0.1	0.06	0.1	6	2
8	0.4	0.2	0.07	0.2	8	3
9	0.1	0.3	0.08	0.3	10	4
10	0.2	0.4	0.09	0.4	6	2
11	0.3	0.5	0.05	0.5	8	3

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import pandas as pd
```

```
# === 1. Генерация обучающих и тестовых данных ===
```

```
def target_function(x, a=0.3, b=0.1, c=0.06, d=0.1):
    return a * torch.cos(b * x) + c * torch.sin(d * x)
```

```
# Параметры
```

```
a, b, c, d = 0.3, 0.1, 0.06, 0.1
```

```
input_size = 6
```

```
hidden_size = 32
```

```
num_samples = 200
```

```
train_ratio = 0.8
```

```
seq_len = 10 # длина окна для RNN
```

```
# === Генерация признаков (степени x) ===
```

```
x = torch.linspace(0, 10, num_samples).reshape(-1, 1)
```

```
X = torch.cat([x ** i for i in range(1, input_size + 1)], dim=1)
```

```
y = target_function(x, a, b, c, d)
```

```
# === Нормализация ===
```

```
X_mean, X_std = X.mean(0, keepdim=True), X.std(0, keepdim=True)
```

```
X_norm = (X - X_mean) / X_std
```

```
# === Формирование последовательностей ===
```

```
def create_sequences(X, y, seq_len):
```

```

Xs, ys = [], []
for i in range(len(X) - seq_len):
    Xs.append(X[i:i + seq_len])
    ys.append(y[i + seq_len])
return torch.stack(Xs), torch.stack(ys)

X_seq, y_seq = create_sequences(X_norm, y, seq_len)

# === Train/Test split ===
split_idx = int(len(X_seq) * train_ratio)
X_train, X_test = X_seq[:split_idx], X_seq[split_idx:]
y_train, y_test = y_seq[:split_idx], y_seq[split_idx:]

print(f'Обучающая выборка: {X_train.shape}, Тестовая: {X_test.shape}')

# === 2. Архитектура LSTM ===
class LSTMModel(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        out, _ = self.lstm(x)
        out = out[:, -1, :] # последний шаг LSTM
        return self.fc(out)

model = LSTMModel(input_size, hidden_size)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

# === 3. Обучение ===
losses = []
epochs = 500

for epoch in range(epochs):
    model.train()
    pred = model(X_train)
    loss = criterion(pred, y_train)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    losses.append(loss.item())

print("Минимальная ошибка:", min(losses))

# === 4. График ошибки ===
plt.figure(figsize=(10, 4))
plt.plot(losses)
plt.title("Ошибка обучения")
plt.xlabel("Эпоха")
plt.ylabel("MSE")
plt.grid(True)
plt.show()

# === 5. Прогноз на обучающей выборке ===
model.eval()
with torch.no_grad():
    y_train_pred = model(X_train)
    y_test_pred = model(X_test)

```

```

# В исходные индексы X
x_train_plot = x[seq_len: seq_len + len(y_train)]
x_test_plot = x[seq_len + len(y_train): seq_len + len(y_train) + len(y_test)]

plt.figure(figsize=(10, 4))
plt.plot(x_train_plot, y_train, label="Эталон")
plt.plot(x_train_plot, y_train_pred, label="Прогноз LSTM")
plt.title("Прогноз на обучающем участке")
plt.grid(True)
plt.legend()
plt.show()

# === 6. Таблица обучения ===
train_table = pd.DataFrame({
    "Эталон": y_train.squeeze().numpy(),
    "Прогноз": y_train_pred.squeeze().numpy(),
    "Отклонение": (y_train_pred - y_train).squeeze().numpy()
})
print(train_table.head())

# === 7. Таблица прогноза ===
test_table = pd.DataFrame({
    "Эталон": y_test.squeeze().numpy(),
    "Прогноз": y_test_pred.squeeze().numpy(),
    "Отклонение": (y_test_pred - y_test).squeeze().numpy()
})
print(test_table.head())

```

Результат:

```

Обучающая выборка: torch.Size([152, 10, 6]), Тестовая: torch.Size([38, 10, 6])
Минимальная ошибка: 3.2738222440953e-07
2025-12-02 00:04:24.570 Python[20473:488295] +[IMKClient subclass]: chose IMKClient_Legacy
2025-12-02 00:04:24.570 Python[20473:488295] +[IMKInputSession subclass]: chose IMKInputSession_Legacy

```

	Эталон	Прогноз	Отклонение
0	0.302635	0.304260	0.001625
1	0.302857	0.304354	0.001498
2	0.303071	0.304445	0.001374
3	0.303277	0.304531	0.001254
4	0.303476	0.304614	0.001139

	Эталон	Прогноз	Отклонение
0	0.249589	0.251149	0.001561
1	0.248696	0.250552	0.001855
2	0.247798	0.249965	0.002167
3	0.246893	0.249389	0.002496
4	0.245982	0.248821	0.002839

